

The Complete Guide to Building a

Library Management System

Built with Django & Django REST Framework

From idea to a complete project — one step at a time

Based on a full source-code walkthrough of the actual Library System project

A hands-on textbook for students and beginner-to-intermediate web developers

Development Team Roster

Member Name	Primary Role	Core Engineering Focus Areas
Doaa Mohamed Adly	Team Leader & System Architect	System Architecture, App Integration, Business Logic & Project Management
Zainab Salama	Backend Developer	Relational Models, Django Admin Customization & Database Integrity Constraints
Huda Ashraf	Backend Developer	Loan State Rules, Validation Logic, Exception Handling & Security Constraints
Ragaa Adel El-Sayed	Frontend Developer	UI/UX Responsive Architecture, Master Templates & Glassmorphism Design System
Mariam Mohamed	Frontend Developer	Interactive Workflows, Live Demo Dashboard, Search Filters & Toast Notifications

Preface

This book is not a theoretical overview — it is a complete, chapter-by-chapter breakdown of a real Library Management System built with Python and Django. The goal is simple: by the end of it, you should understand every meaningful line of code in the project, why it was written that way, and how to build a system like it yourself, from scratch.

We will walk through the project together starting from the very first moment the project is created, through the data models and database, the admin panel, the views and templates, the REST API, the authentication system, and finally, the steps needed to run the whole project on your own machine.

Note:

The reference project used throughout this book is called "Smart Library" — a complete Django system made up of three main apps: catalog (the book catalog), loans (borrowing and returning), and accounts (signup, login, and logout) — plus a REST API built with Django REST Framework.

Who is this book for?

- A student with basic Python knowledge and some web theory who wants to see how that theory turns into a working project.
- A beginner Django developer who wants to understand how Models, Views, Templates, and URLs actually connect to each other.
- Anyone who wants a reference while building a system with relationships between tables (books – users – loans), validation rules, and an API.

What will you be able to do after finishing this book?

- Fully understand the structure of a professional Django project split across multiple apps.
- Build an authentication system (signup / login / logout) using Django's built-in tools.
- Build Models with relationships (ForeignKey) and custom validation rules (the clean method).
- Build a simple REST API with Django REST Framework and connect it to an HTML front end.
- Customize the Django Admin panel with custom actions and displays.
- Run the whole project locally and seed it with sample data.

Table of Contents

2		Preface
4		Chapter 1: Project Overview
5		Chapter 2: Project Architecture
7		Chapter 3: Prerequisites and Environment Setup
8		Chapter 4: Creating the Project Step by Step
9		Chapter 5: The settings.py File in Depth
11		Chapter 6: The catalog App — the Book Model
13		Chapter 7: Customizing the Book Admin Panel
15		Chapter 8: Building the API — Serializers
16		Chapter 9: Displaying Books — Views and URLs
18		Chapter 10: The loans App — the Loan Model
20		Chapter 11: Borrowing and Returning — Views
23		Chapter 12: Wiring Up Loan URLs (loans/urls.py)
24		Chapter 13: The accounts App — Login and Signup
26		Chapter 14: Bringing It All Together — Project URLs
27		Chapter 15: Templates
30		Chapter 16: Static Files, Media, and Sample Data
32		Chapter 17: Running the Project End to End
33		Chapter 18: Quick API Reference
35		Chapter 19: The Complete User Journey
36		Chapter 20: Notes and Suggested Improvements
38		Conclusion

Chapter 1: Project Overview

Before opening a single file, we need to be clear on exactly what we're building. The project is a "Library Management System" whose core purpose is to let a library — school, university, or public — manage its books and members entirely online.

Core features of the system

- Browse the library's book catalog, with search by title or author.
- Register a new member account, and log in / log out.
- Borrow an available book (if copies remain), while preventing borrowing once stock runs out.
- View a "My Loans" page for each member, and return a book once finished.
- An admin dashboard for adding new books, adjusting copy counts, and tracking each book's status.
- A REST API exposing the same data and operations to any external client (a mobile app, for instance).

Why Django?

Django is a Python framework for building web applications quickly and securely. It follows a design pattern called MVT (Model - View - Template), which is Django's take on the well-known MVC pattern:

Layer	Responsibility
Model	The data layer; represents database tables and the validation rules around them.
View	The logic layer; receives the Request, decides what should happen, and returns a Response.
Template	The presentation layer; HTML files that render data for the user.

Django ships with a lot out of the box: a user authentication system, an automatic admin panel, an ORM for talking to the database without hand-written SQL, and a migrations system for tracking database changes over time. That's exactly what makes it a strong choice for a project like this one.

Note:

We'll also use Django REST Framework (DRF) to build an API alongside the regular HTML pages — so the same project serves both browsers and external applications at once.

Chapter 2: Project Architecture

Django encourages splitting a project into small "apps", each responsible for one clearly defined part of the system. Our project has one main project, `library_system`, containing three apps:

App name	Responsibility
accounts	Everything related to users: signup, login, logout.
catalog	Everything related to books: the Book model, the catalog page, search, and a books API.
loans	Everything related to borrowing and returning: the Loan model, and a borrowing API.

What the project tree looks like

```

library_system-main/
├── library_system/          # Main project settings (settings, urls, wsgi)
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py / asgi.py
├── accounts/               # Accounts app (signup / login / logout)
│   ├── views.py
│   ├── urls.py
│   └── templates/accounts/
├── catalog/               # Catalog app (books)
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── admin.py
│   └── urls.py
├── loans/                 # Borrowing and returning app
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
├── templates/             # Shared HTML templates (base, home, ...)
│   └── static/             # CSS and JS files
├── media/covers/          # Uploaded book cover images
├── populate_books.py       # Script to seed the database with sample data
└── manage.py               # Project entry point

```

The request flow inside Django

To understand how these pieces work together, imagine a user opening the book catalog page in their browser. The journey looks like this:

1. The browser sends a request to a particular URL, for example `/catalog/`.

2. The main `library_system/urls.py` file receives the request and hands it off to catalog's own `urls.py`.
3. `catalog/urls.py` determines that this URL should be routed to a function in `views.py` called `book_list_html`.
4. The `book_list_html` function talks to the Book model to fetch book data from the database.
5. The function renders the `catalog/book_list.html` template and passes it the data it fetched.
6. Django turns the template into a full HTML page and returns it to the browser as the response.

Note:

Once you understand this flow, you'll notice every app in the project (catalog, loans, accounts) repeats the same pattern: Model → View → Template/Serializer → URL. That pattern is the foundation for understanding almost any Django project.

Chapter 3: Prerequisites and Environment Setup

Before we start implementing anything, we need to prepare our working environment. These steps are only done once, at the very beginning of any Django project.

1. Install Python

The project requires Python 3.10 or later. You can check your installed version with:

```
python --version
# or on some systems:
python3 --version
```

2. Create a virtual environment

A virtual environment isolates your project's dependencies from every other Python project on your machine — this is standard best practice for any Python project:

```
# create the project folder and move into it
mkdir library_system && cd library_system

# create the virtual environment
python -m venv venv

# activate it on Windows
venv\Scripts\activate

# activate it on macOS / Linux
source venv/bin/activate
```

3. Install the required packages

The project relies on four core packages: Django itself, Django REST Framework for the API, drf-spectacular for automatic API documentation, and Pillow for handling uploaded book cover images (ImageField).

```
pip install django djangorestframework drf-spectacular pillow
```

Warning:

The original project doesn't ship a ready-made requirements.txt file. After installing the packages, it's good practice to generate one yourself so anyone else can recreate the same environment: `pip freeze > requirements.txt`

Chapter 4: Creating the Project Step by Step

Now we begin the actual implementation. We'll build the basic project skeleton using Django's official commands.

1. Create the main project

```
django-admin startproject library_system .  
# the trailing "." creates the project files in the current folder  
# instead of nesting them inside an extra subfolder
```

This command generates a `library_system` folder containing the project settings (`settings.py`, `urls.py`, `wsgi.py`, `asgi.py`), plus a `manage.py` file at the root — the file we'll use for every management command from here on.

2. Create the three apps

```
python manage.py startapp accounts  
python manage.py startapp catalog  
python manage.py startapp loans
```

Each of these commands generates a new app folder pre-populated with files like `models.py`, `views.py`, `admin.py`, `apps.py`, and a `migrations` folder. We'll fill these files in gradually over the coming chapters.

3. Register the apps with the project

Any app you create in Django needs to be "registered" with the project before it will do anything, and that happens through the `INSTALLED_APPS` list inside `settings.py` — our first edit to that file, coming up next.

4. Create the templates and static folders

Since we have templates shared across apps (like `base.html` and `home.html`), we create a `templates` folder at the project root, plus a `static` folder for CSS and JS, and a `media` folder for uploaded files such as book covers:

```
mkdir templates  
mkdir static  
mkdir -p media/covers
```

Note:

Notice that the `accounts` templates (`login` and `signup`) live inside `accounts/templates/accounts/` — meaning the `accounts` app keeps its own templates local to itself, while shared templates like `home.html` and `base.html` live in the project-level `templates/` folder. Django supports both approaches; which one you choose is a matter of personal project organization.

Chapter 5: The settings.py File in Depth

settings.py is the heart of any Django project — it defines everything: which apps are active, the database, where templates live, the language, and more. Let's walk through the parts we need to edit, in order.

1. Enabling the apps (INSTALLED_APPS)

We need to add our three apps, plus two third-party packages: rest_framework and drf_spectacular:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # API library
    'rest_framework',
    'drf_spectacular',

    # project apps
    'accounts',
    'catalog',
    'loans',
]
```

2. Template path (TEMPLATES)

For Django to find the shared templates we placed in the root-level templates/ folder (not inside any specific app), we need to add its path under DIRS:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'], # path to the main templates folder
        'APP_DIRS': True, # tells Django to also look inside each app (e.g.
                           # accounts/templates)
        ...
    },
]
```

3. Database (DATABASES)

The project uses SQLite, which is a great fit for learning and local development because it's just a single file, with no extra server setup required:

```
DATABASES = {
    'default': {
```

```
'ENGINE': 'django.db.backends.sqlite3',
'NAME': BASE_DIR / 'db.sqlite3',
}
```

4. Static files and media

We need to distinguish between two kinds of files: static files (CSS, JS), and media — files uploaded by a user or admin, such as book cover images:

```
STATIC_URL = 'static/'
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]

import os
MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

5. Login and logout redirect URLs

```
LOGIN_REDIRECT_URL = 'catalog:book_list' # send the user to the catalog after login
LOGOUT_REDIRECT_URL = '/' # send the user back home after logout
```

Warning:

In the original settings file, you'll find `LOGIN_REDIRECT_URL` and `LOGOUT_REDIRECT_URL` defined twice in different places (the second value simply overrides the first). This works fine — Python just keeps the last value — but it's a good example of small "technical debt": it's best practice to group related settings together in one place so the file stays easy to read and maintain.

6. An important security note (SECRET_KEY and DEBUG)

In development mode, Django auto-generates a `SECRET_KEY` and sets `DEBUG = True`. Both are fine for development but genuinely dangerous in a production environment:

- `SECRET_KEY`: should never be hard-coded into the source; it belongs in an environment variable before the project ever reaches a real server.
- `DEBUG`: must be `False` in production — otherwise, if an error happens, users can see detailed internal code and server paths.
- `ALLOWED_HOSTS`: in production, this must list the exact domain names the project is allowed to run on.

Chapter 6: The catalog App — the Book Model

We'll start by building the heart of the catalog: the Book model. A Django model is a Python class representing a database table, where each attribute becomes a column in that table.

Writing the model in catalog/models.py

```
from django.db import models
from django.core.exceptions import ValidationError

class Book(models.Model):
    title = models.CharField(max_length=200, verbose_name="Title")
    author = models.CharField(max_length=100, verbose_name="Author")
    isbn = models.CharField(max_length=13, unique=True, verbose_name="ISBN")
    category = models.CharField(max_length=50, verbose_name="Category")
    total_copies = models.PositiveIntegerField(default=1, verbose_name="Total
Copies")
    available_copies = models.PositiveIntegerField(default=1, verbose_name="Available
Copies")
    cover = models.ImageField(upload_to='covers/', blank=True, null=True)
    cover_image = models.ImageField(upload_to='book_covers/', blank=True, null=True)

    class Meta:
        verbose_name = "Book"
        verbose_name_plural = "Books"
        ordering = ['title']

    def clean(self):
        if self.available_copies > self.total_copies:
            raise ValidationError(
                {'available_copies': 'Available copies cannot be greater than total
copies!'}
            )

    def __str__(self):
        return f"{self.title} - {self.author}"
```

Field by field

Field	Purpose
title / author / category	Short text (CharField); max_length is required.
isbn	The book's international number; unique=True stops the same book being registered twice.
total_copies / available_copies	PositiveIntegerField for the total and available copy counts, defaulting to 1.
cover / cover_image	ImageField for the cover photo; blank=True, null=True makes them optional.

The Meta class

The inner Meta class is not a database field — it configures the model's behavior. `verbose_name` is the label shown in the admin panel, and `ordering = ['title']` means any query on books is automatically sorted alphabetically by title, without repeating that logic in every View.

The `clean()` method — validating the data

This is the most important detail in the model. `clean()` runs automatically (when used) before saving data through Forms or the admin panel, and here we use it to prevent an illogical state: `available_copies` being greater than `total_copies` (you can't have 10 copies available for a book that only has 5 copies total, for example).

Warning:

`clean()` runs automatically when you use a `ModelForm` or the admin panel, but it does NOT run automatically if you call `Book.objects.create()` directly from code. If you want it enforced everywhere, you need to call `instance.full_clean()` manually before `.save()`.

Generating and applying the migration

After writing or editing any model, Django needs to "translate" that change into actual SQL commands against the database, which happens in two steps:

```
python manage.py makemigrations catalog
python manage.py migrate
```

The first command generates a new migration file (like `0001_initial.py`) describing the change you made, and the second actually applies it to the database and creates the `Book` table.

Chapter 7: Customizing the Book Admin Panel

Django gives you a ready-made Admin Panel the moment you register a model with it. In our project, though, that panel has been customized significantly, to make it easier and clearer for a library manager to use.

Registering the base model

```
from django.contrib import admin
from .models import Book

@admin.register(Book)
class BookAdmin(admin.ModelAdmin):
    list_display = ('cover_preview', 'title', 'author', 'category',
                    'total_copies', 'available_copies', 'status_badge')
    list_editable = ('available_copies',)
    search_fields = ('title', 'author', 'isbn')
    search_help_text = "Search by Title, Author, or ISBN"
    list_filter = ('category',)
    ordering = ('title',)
    list_per_page = 10
    save_on_top = True
```

Attribute	Purpose
<code>list_display</code>	The columns shown in the book list table inside the panel.
<code>list_editable</code>	Makes the <code>available_copies</code> column editable directly from the table, without opening the book.
<code>search_fields</code>	The fields the search box at the top of the page will search.
<code>list_filter</code>	Adds a sidebar filter based on category.
<code>list_per_page</code>	How many rows are shown per page (pagination).

Grouping fields (fieldsets)

Instead of letting every book field appear in a random order on the add/edit page, we can group them into logical sections, each with its own heading:

```

        fieldsets = (
            ('General Information', {
                'fields': ('title', 'author', 'isbn', 'category', 'cover')
            }),
            ('Inventory Management', {
                'fields': ('total_copies', 'available_copies'),
            }),
        )

```

Custom columns (custom display methods)

Notice that `list_display` includes `cover_preview` and `status_badge`, which aren't real fields on the model — they're methods on `BookAdmin` that return custom HTML:

```

        @admin.display(description='Cover')
        def cover_preview(self, obj):
            if obj.cover:
                return mark_safe(
                    f''
                )
            return mark_safe('<span style="color: #999;">No Cover</span>')

        @admin.display(description='Status')
        def status_badge(self, obj):
            if obj.available_copies > 0:
                return mark_safe('<span style="...">Available</span>')
            return mark_safe('<span style="...">Out of Stock</span>')

```

Warning:

`mark_safe()` tells Django "trust this string as safe HTML — render it without escaping." Use it very carefully, and only with strings you wrote yourself in the code — never with text coming directly from a user, or you open the door to an XSS vulnerability.

Custom actions

The panel also includes two custom actions that appear in the dropdown above the book table, letting the library manager bump the copy count for a batch of books up or down at once:

```

actions = ['add_one_copy', 'remove_one_copy']

@admin.action(description='Add 1 copy to selected books')
def add_one_copy(self, request, queryset):
    for book in queryset:
        book.total_copies += 1
        book.available_copies += 1
        book.save()
    self.message_user(request, "Successfully added 1 copy!")

```

Here, `queryset` represents every book the admin checked, and we loop over them, updating and saving each one.

Creating an admin user and trying the panel

```
python manage.py createsuperuser
# it will ask for a username, email, and password
python manage.py runserver
# open your browser at: http://127.0.0.1:8000/admin/
```


Chapter 8: Building the API — Serializers

So far we've built the model and managed it from the admin panel. But if we want a mobile app or any external system to read book data, we need an API that returns data as JSON instead of HTML.

This is where Django REST Framework comes in.

What is a Serializer?

A Serializer is responsible for converting a Python object — like a `Book` instance coming from the database — into JSON that can be sent over the network, and back again (turning JSON from an incoming request into a Python object that can be saved to the database).

Writing BookSerializer

```
# catalog/serializers.py
from rest_framework import serializers
from .models import Book

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = '__all__'
```

Using `ModelSerializer` saves a lot of time: we don't need to declare every field by hand, because DRF reads the model's fields automatically and generates matching serializer fields. `fields = '__all__'` means "include every model field in the resulting JSON."

Note:

In a larger real-world project, you would usually list fields explicitly instead of using `__all__` (e.g. `fields = ['id', 'title', 'author', 'available_copies']`), so you have full control over what the API exposes and never accidentally leak a sensitive field.

Chapter 9: Displaying Books — Views and URLs

Now that we have a model and a serializer, we need the Views that connect them to the user, whether through an HTML page or a JSON response.

The home View

```
# catalog/views.py
def home(request):
    total_books = Book.objects.count()
    available_copies = sum(book.available_copies for book in Book.objects.all())
    active_loans = 0
    if request.user.is_authenticated:
        active_loans = Loan.objects.filter(
            member=request.user, return_date__isnull=True
        ).count()

    recent_books = Book.objects.order_by('-id')[:3]

    context = {
        'total_books': total_books,
        'available_copies': available_copies,
        'active_loans': active_loans,
        'recent_books': recent_books,
    }
    return render(request, 'home.html', context)
```

This function computes a few simple statistics (total books, total available copies, the current user's active loans), fetches the three most recently added books, and passes all of it to the `home.html` template through a context dictionary — the template is what actually renders them visually.

The catalog View (HTML) and search

```
def book_list_html(request):
    query = (request.GET.get('query') or request.GET.get('q')
            or request.GET.get('search') or '').strip()
    books = Book.objects.all()

    if query:
        books = books.filter(
            Q(title__icontains=query) | Q(author__icontains=query)
        )

    return render(request, 'catalog/book_list.html', {
        'books': books,
        'query': query,
    })
```

Here we used Django's Q objects to build an OR condition: fetch any book whose title contains the search text, OR whose author contains it. `icontains` means the match is case-insensitive and can occur anywhere in the value.

Note:

Notice the function accepts more than one possible parameter name (query, q, search) — extra flexibility so that no matter which name a link or search form in the template uses, the View still works correctly.

The books API View

```
class BookListView(generics.ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    filter_backends = [filters.SearchFilter]
    search_fields = ['title', 'author', 'category']
```

Here we used a ready-made DRF class-based view, `ListAPIView`, whose sole job is to return a list of items as JSON. By simply setting `queryset` and `serializer_class`, the View works fully without writing a single line of manual logic. We also added `SearchFilter` so it supports search via `?search=` in the URL.

Wiring up the URLs (catalog/urls.py)

```
from django.urls import path
from .views import BookListView, book_list_html
from . import views

app_name = 'catalog'
urlpatterns = [
    path('', views.book_list_html, name='book_list'),
    path('api/books/', BookListView.as_view(), name='api_books'),
]
```

`app_name = 'catalog'` defines a namespace for this app, which lets us write `{% url 'catalog:book_list' %}` anywhere in the project without conflicting with a URL of the same name (`book_list`) defined in a different app.

URL	Method	Purpose
<code>/catalog/</code>	GET	An HTML page listing every book, with search support via <code>?q=</code>
<code>/catalog/api/books/</code>	GET	A JSON response listing every book, with search support via <code>?search=</code>

Chapter 10: The loans App — the Loan Model

Now we get to the most important piece of business logic in the whole project: the borrowing system. Here we need a model that links a User to a Book, which means a relationship between three tables.

Writing the Loan model

```
# loans/models.py
from django.db import models
from django.core.exceptions import ValidationError
from django.contrib.auth.models import User
from django.utils import timezone
from datetime import timedelta
from catalog.models import Book

class Loan(models.Model):
    STATUS_CHOICES = (
        ('BORROWED', 'Borrowed'),
        ('RETURNED', 'Returned'),
    )

    member = models.ForeignKey(User, on_delete=models.CASCADE,
                              related_name='loans', verbose_name="Member")
    book = models.ForeignKey(Book, on_delete=models.CASCADE,
                             related_name='loans', verbose_name="Book")
    borrow_date = models.DateTimeField(auto_now_add=True)
    return_date = models.DateTimeField(null=True, blank=True)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES,
                              default='BORROWED')

    class Meta:
        ordering = ['-borrow_date']
```

Understanding ForeignKey and related_name

Every Loan record is linked to exactly one user (member) and one book (book) through a ForeignKey — a "one-to-many" relationship: a single user can have many loans, and a single book can be borrowed many times over.

Element	Meaning
on_delete=models.CASCADE	If the related user or book is deleted, every loan record linked to it is deleted automatically too.
related_name='loans'	Lets you reach every loan from a User or Book instance by writing user.loans.all() or book.loans.all().
auto_now_add=True	Records the creation timestamp automatically, once, and it can never be changed afterward.
choices=STATUS_CHOICES	Restricts the status field to exactly two values: BORROWED or RETURNED.

The clean() method — business rules

This is where we find the most important logic in the entire project: validating that a new loan is actually allowed.

```
def clean(self):
    if not self.pk and self.book.available_copies <= 0:
        raise ValidationError(
            f"Cannot process loan: '{self.book.title}' has zero available copies!"
        )

    if not self.pk:
        fourteen_days_ago = timezone.now() - timedelta(days=14)
        has_overdue = Loan.objects.filter(
            member=self.member,
            status='BORROWED',
            borrow_date__lt=fourteen_days_ago
        ).exists()

        if has_overdue:
            raise ValidationError(
                f"Cannot process loan: Member '{self.member.username}' has "
                "overdue books that must be returned first!"
            )
```

Both rules only apply when creating a brand-new record (not self.pk means the record has no primary key yet — it hasn't been saved to the database):

7. You cannot borrow a book that has zero available copies.
8. You cannot borrow a new book if you already have an overdue book (borrowed more than 14 days ago) that hasn't been returned.

Note:

`timedelta(days=14)` and `timezone.now()` are the correct way to work with dates and times in Django (rather than plain `datetime.now()`), because they respect the timezone configured via `USE_TZ = True` in the settings.

Chapter 11: Borrowing and Returning — Views

The loans app has two kinds of Views, just like catalog: regular Views that return HTML for browser pages, and an APIView that returns JSON.

The borrow_book View (HTML)

```
@login_required
def borrow_book(request, book_id):
    if request.method == 'POST':
        book = get_object_or_404(Book, id=book_id)

        if Loan.objects.filter(member=request.user, book=book,
                               status='BORROWED').exists():
            messages.warning(request, f"You've already borrowed '{book.title}'!")
            elif book.available_copies <= 0:
                messages.error(request, "Sorry, this book isn't available right now.")
            else:
                Loan.objects.create(member=request.user, book=book)
                book.available_copies -= 1
                book.save()
            messages.success(request, f"You've successfully borrowed
                                '{book.title}'!")
            return redirect('loans:my_loans')

    return redirect('catalog:book_list')
```

Element	Purpose
@login_required	A decorator that blocks any visitor who isn't logged in from reaching the function; automatically redirects them to the login page.
get_object_or_404	Fetches the book by its id, and automatically returns a 404 error page if it doesn't exist, instead of crashing.
messages.success / warning / error	Django's built-in messages framework; lets you send the user a one-time flash message shown on the next page.

Notice the order of operations in the borrowing flow: first we check the user hasn't already borrowed this exact book, then that copies are actually available, and only then do we create a new

Loan record and decrement `available_copies`. Both of those last two steps happen together, so the data stays consistent.

The `return_book` View

```
@login_required
def return_book(request, loan_id):
    if request.method == 'POST':
        loan = get_object_or_404(
            Loan, id=loan_id, member=request.user, status='BORROWED'
        )
        loan.status = 'RETURNED'
        loan.return_date = timezone.now()
        loan.save()

        loan.book.available_copies += 1
        loan.book.save()

    return redirect('loans:my_loans')
```

Warning:

Notice that `get_object_or_404` here requires `member=request.user` — this matters a lot for security: it stops any user from returning a book they didn't personally borrow, even if they somehow know someone else's `loan_id`.

The "My Loans" View

```
@login_required
def my_loans(request):
    loans = Loan.objects.filter(member=request.user, status='BORROWED')
    return render(request, 'loans/my_loans.html', {'loans': loans})
```

The borrowing API (`BorrowBookAPIView`)

This is the programmatic version of the same borrowing operation, but it returns structured JSON responses instead of HTML pages and instead of the messages framework — which is exactly what makes it usable from something like a mobile app:

```
class BorrowBookAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request, book_id):
        try:
            book = Book.objects.get(id=book_id)
        except Book.DoesNotExist:
            return Response({
                "status": "error", "message": "Book not found.",
                "code": "NOT_FOUND"
            }, status=status.HTTP_404_NOT_FOUND)
```

```

        if book.available_copies <= 0:
            return Response({
                "status": "error",
                "message": f"Loan Failed: '{book.title}' has zero available copies.",
                "code": "OUT_OF_STOCK"
            }, status=status.HTTP_400_BAD_REQUEST)
        # ... the same check for a pre-existing loan is repeated here
        loan = Loan.objects.create(member=request.user, book=book)
        book.available_copies -= 1
        book.save()

        return Response({
            "status": "success",
            "message": f"Successfully borrowed '{book.title}'!",
            "data": LoanSerializer(loan).data
        }, status=status.HTTP_201_CREATED)

```

`permission_classes = [IsAuthenticated]` is DRF's version of the same idea as `@login_required`, but at the API level: any request without a valid session or token is automatically rejected with a 401 response before it ever reaches the function's code.

The Loan Serializer

```

# loans/serializers.py
class LoanSerializer(serializers.ModelSerializer):
    book_title = serializers.ReadOnlyField(source='book.title')
    member_username = serializers.ReadOnlyField(source='member.username')

    class Meta:
        model = Loan
        fields = ['id', 'member', 'member_username', 'book', 'book_title',
                  'borrow_date', 'return_date', 'status']

```

`book_title` and `member_username` are "computed" fields that don't exist directly on the `Loan` model, but are pulled through the relationship (`source='book.title'`) so the API response is more useful — instead of just returning `book: 3`, it also returns the actual book title.

Chapter 12: Wiring Up Loan URLs (loans/urls.py)

```

from django.urls import path
from .views import BorrowBookAPIView, my_loans, return_book
from . import views

app_name = 'loans'
urlpatterns = [
    path('api/borrow/<int:book_id>/', BorrowBookAPIView.as_view(),
          name='api_borrow_book'),
    path('my-loans/', my_loans, name='my_loans'),
    path('return/<int:loan_id>/', return_book, name='return_book'),
    path('borrow/<int:book_id>/', views.borrow_book, name='borrow_book'),
]

```

`<int:book_id>` is a "path converter": it tells Django "take this segment of the URL, convert it to an integer, and pass it to the function as a parameter called `book_id`." If someone puts letters instead of digits here, Django rejects it automatically with a 404, with no extra validation code needed from us.

URL	Method	Purpose
/loans/borrow/<id>/	POST	Borrow a book (HTML, requires login)
/loans/return/<id>/	POST	Return a borrowed book (HTML)
/loans/my-loans/	GET	View the list of books the user currently has on loan
/loans/api/borrow/<id>/	POST	Borrow a book through the API — returns JSON

Exercise:

An important observation: the loans app has an API endpoint for borrowing, but no matching API endpoint for returning (e.g. `api/return/`). If you want to build a complete mobile app on top of this project, that's the first thing you'd need to add — a good exercise to try at the end of this book.

Chapter 13: The accounts App — Login and Signup

The accounts app is the simplest app in the project because it leans heavily on Django's built-in authentication system (`django.contrib.auth`) — meaning we don't need to build login from scratch, just customize it lightly.

Signing up (Sign Up)

```
# accounts/views.py
from django.contrib.auth.forms import UserCreationForm

def signup_view(request):
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, 'Account created successfully. Please log in.')
            return redirect('accounts:login')
        else:
            form = UserCreationForm()
    return render(request, 'accounts/signup.html', {'form': form})
```

`UserCreationForm` is a ready-made Django Form that includes username and password fields (with confirmation), and automatically checks the password is strong enough according to the `AUTH_PASSWORD_VALIDATORS` defined in `settings.py`.

Login and logout

```
from django.contrib.auth.views import LoginView, LogoutView

class CustomLoginView(LoginView):
    template_name = 'accounts/login.html'

class CustomLogoutView(LogoutView):
    next_page = 'login'
```

Instead of writing login logic by hand (checking username/password, creating a session, etc.), we use Django's ready-made `LoginView` and `LogoutView`, and simply subclass them to point at our own template.

The accounts URLs

```
app_name = 'accounts'
urlpatterns = [
    path('signup/', views.signup_view, name='signup'),
    path('login/', views.CustomLoginView.as_view(), name='login'),
    path('logout/', views.CustomLogoutView.as_view(), name='logout'),
]
```

Warning:

The project registers both its own accounts URLs (accounts/login/) and Django's built-in django.contrib.auth.urls (which also defines its own login/ path) — both under accounts/ in the root urls.py. It's best to settle on one or the other going forward, to avoid any naming conflicts or confusion later.

Chapter 14: Bringing It All Together — Project URLs

After building each app's own URLs, we need to wire them all together at a single entry point:

library_system/urls.py.

```

        from django.contrib import admin
        from django.urls import path, include
        from catalog import views as catalog_views
        from django.conf import settings
        from django.conf.urls.static import static

        urlpatterns = [
            path('admin/', admin.site.urls),

            # home page
            path('', catalog_views.home, name='home'),
            path('accounts/', include(('accounts.urls', 'accounts'), namespace='accounts')),

            # the rest of the app routes
            path('catalog/', include('catalog.urls')),
            path('loans/', include('loans.urls')),
            path('accounts/', include('django.contrib.auth.urls')),
        ]

        if settings.DEBUG:
            urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

```

`include()` is the function that lets us "mount" an entire sub-app's `urls.py` under a given path prefix, instead of writing every route in the entire project inside a single, enormous file. Each app becomes responsible for its own routes, which lets the project scale cleanly.

Why the last two lines matter

If `settings.DEBUG` is `True` (meaning we're in development mode), we add a special route that lets Django itself serve uploaded files from the media folder directly, so book cover images actually show up in the browser while we're developing.

Note:

In a real production environment, these lines are removed, and serving media files becomes the responsibility of a dedicated server like Nginx or a cloud storage service (e.g. S3) — Django itself isn't built to efficiently serve static files at scale.

Chapter 15: Templates

Templates are the presentation layer of the project — plain HTML files, but with a small templating language called the Django Template Language (DTL) mixed in, letting you embed variables and simple logic (conditionals, loops) directly inside the HTML.

The base template, base.html

base.html is the "skeleton" every page on the site inherits from: it holds the navbar, the footer, and the toast notification system, plus an empty block called content that each individual page fills in:

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>{% block title %}Library Management System{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
  </head>
  <body>
    <nav> ... navigation and search bar ... </nav>

    <div class="container">
      {% block content %}
      {% endblock %}
    </div>

    <div class="toast-container">
      {% if messages %}
      {% for message in messages %}
      <div class="toast-card {{ message.tags }}">{{ message }}</div>
      {% endfor %}
      {% endif %}
    </div>

    <footer> ... </footer>
  </body>
</html>
```

Element	Purpose
{% load static %}	Enables {% static %} for loading CSS/JS/images from the static folder.
{% block title %} / {% block content %}	"Slots" any child template can fill in with different content.
{% if user.is_authenticated %}	A condition checking whether the user is logged in, so we can show different nav

	links (Login vs. My Books/Logout).
{% if messages %} ... {% endfor %}	Loops over every message sent via messages.success/error in the Views and displays it as a toast.

Template inheritance

Every other template in the project (home.html, book_list.html, my_loans.html) starts with the same line:

```
{% extends 'base.html' %}
{% load static %}

{% block title %}Home - Library Management System{% endblock %}

{% block content %}
<!-- this page's own content goes here -->
{% endblock %}
```

{% extends %} says "take all of base.html's structure, and only replace the title and content blocks with what I write here." That way, the navbar, footer, and toast system never need to be repeated in every file — they're written once, in base.html.

The catalog page, book_list.html

This template receives two variables from the View: books (the list of books) and query (the current search term, if any), then loops over books rendering each one as a card, alongside a search form at the top that sends a GET request to the same URL:

```
<form action="{% url 'catalog:book_list' %}" method="GET">
<input type="search" name="q" value="{{ query }}" placeholder="Search">
<button type="submit">Search</button>
</form>

{% for book in books %}
<div class="book-card">
<h3>{{ book.title }}</h3>
<p>{{ book.author }}</p>
{% if book.available_copies > 0 %}
<form action="{% url 'loans:borrow_book' book.id %}" method="POST">
    {% csrf_token %}
    <button type="submit">Borrow</button>
</form>
{% endif %}
</div>
{% empty %}
<p>No books found.</p>
{% endfor %}
```

Warning:

{% csrf_token %} is required on any form using method="POST" in Django — it generates a hidden security token that verifies the request genuinely came from your own site, and not from a malicious site tricking a user's browser into performing the action on their behalf without their knowledge (an attack known as CSRF).

The "My Loans" page, my_loans.html

Displays the list of loans passed by the View, and each item includes a "Return" button that sends a POST request to the loans:return_book URL.

Chapter 16: Static Files, Media, and Sample Data

Static files

The static/ folder holds everything that never changes based on data: the style.css file with the entire site's visual design (colors, cards, buttons, navbar), and project.js, which adds a few small interactive touches, like instant filtering on the book list page.

```
// static/project.js (excerpt)
const searchInput = document.getElementById("searchInput");
const books = document.querySelectorAll(".book");

searchInput.addEventListener("input", function () {
  const searchtext = searchInput.value.trim().toLowerCase();
  books.forEach(function (book) {
    const booktext = book.textContent.toLowerCase();
    book.style.display = booktext.includes(searchtext) ? "" : "none";
  });
});
```

Note:

Notice the difference between this search and the one covered in Chapter 9: the search in project.js runs entirely in the browser's JavaScript (client-side), filtering cards that are already on the page, without sending any new request to the server. The search inside book_list_html, on the other hand, is server-side, returning fresh results straight from the database. The project supports both at the same time.

Media — book cover images

The media/covers/ folder stores images uploaded to the Book model's cover field. As noted in Chapter 6, the model actually has two image fields: cover (uploaded to media/covers/) and cover_image (uploaded to media/book_covers/) — a bit of unnecessary duplication we'll revisit in the improvements chapter.

Seeding sample data — populate_books.py

Rather than adding every book manually through the admin panel while testing, a standalone script adds a batch of ready-made books and links them to their cover images:

```
import os, django
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'library_system.settings')
django.setup()

from catalog.models import Book

books_data = [
    {"title": "Clean Code", "author": "Robert C. Martin", ...},
    {"title": "Python Crash Course", "author": "Eric Matthes", ...},
    # ... the rest of the books
```



```
    ]

    for item in books_data:
        # looks for any image in media/covers matching the book's title
        book, created = Book.objects.get_or_create(
            title=item["title"],
            defaults={
                "author": item["author"], "category": item["category"],
                "isbn": item["isbn"], "total_copies": 5,
                "available_copies": 5, "cover": matched_file
            }
        )
```

The first two lines matter a lot: since this script isn't a View inside Django (it isn't run through `manage.py runserver` in the normal way), we need to manually "boot" the Django environment before importing any Model — which is exactly what `os.environ.setdefault` and `django.setup()` do.

`get_or_create()` is very useful here: it looks for a record matching the given criteria (here, title); if found, it returns it without doing anything; if not, it creates one using the values in defaults. This prevents the same book being duplicated if you accidentally run the script more than once.

Running the script

```
python populate_books.py
```

Chapter 17: Running the Project End to End

We've reached the decisive moment. Let's now bring together everything we've learned into a single sequence of commands, so you can take the project and run it on your own machine from start to finish.

9. Activate the virtual environment (if it isn't already active):

```
source venv/bin/activate # or venv\Scripts\activate on Windows
```

10. Install the required packages:

```
pip install django djangorestframework drf-spectacular pillow
```

11. Prepare the database by applying migrations:

```
python manage.py makemigrations
python manage.py migrate
```

12. Create an admin (superuser) account so you can access the admin panel:

```
python manage.py createsuperuser
```

13. (Optional) Seed the database with sample data:

```
python populate_books.py
```

14. Start the local development server:

```
python manage.py runserver
```

15. Open your browser and try the following links:

Page	URL
Home page	http://127.0.0.1:8000/
Book catalog	http://127.0.0.1:8000/catalog/
Sign up	http://127.0.0.1:8000/accounts/signup/
Log in	http://127.0.0.1:8000/accounts/login/
Admin panel	http://127.0.0.1:8000/admin/
Books API	http://127.0.0.1:8000/catalog/api/books/

Note:

Page	URL
	If you see a duplicate-ISBN error when running <code>populate_books.py</code> more than once, that's expected — it's caused by <code>unique=True</code> on the <code>isbn</code> field. Use <code>get_or_create()</code> as covered earlier: it will simply skip books that already exist instead of trying to re-add them.

Chapter 18: Quick API Reference

The table below summarizes every endpoint available in the project, as a quick reference while developing or testing the API (with a tool like Postman, or directly in the browser for GET requests).

URL	Method	Login required?	Description
/catalog/api/books/	GET	No	List of all books as JSON; supports ?search=keyword
/loans/api/borrow/<book_id>/	POST	Yes	Borrow a specific book for the current user

Sample response for GET /catalog/api/books/

```
[
  {
    "id": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "978-0132350884",
    "category": "Software Engineering",
    "total_copies": 5,
    "available_copies": 4,
    "cover": "/media/covers/clean_code.jpeg",
    "cover_image": null
  }
]
```

Sample response for POST /loans/api/borrow/1/ (success)

```
// Status: 201 Created
{
  "status": "success",
  "message": "Successfully borrowed 'Clean Code'!",
  "data": {
    "id": 7,
    "member": 3,
    "member_username": "ahmed",
    "book": 1,
    "book_title": "Clean Code",
    "borrow_date": "2026-09-04T10:15:00Z",
    "return_date": null,
    "status": "BORROWED"
  }
}
```

Sample failure response (book unavailable)

```
// Status: 400 Bad Request
{
```

```
"status": "error",  
"message": "Loan Failed: 'Clean Code' has zero available copies in stock.",  
"code": "OUT_OF_STOCK"  
}
```

Exercise:

The project has `drf_spectacular` installed (a library that auto-generates OpenAPI/Swagger API documentation), but no URL is currently registered for it in `urls.py`. If you'd like a self-generating, interactive Swagger-style documentation page, that's an excellent exercise — see the details in the upcoming improvements chapter.

Chapter 19: The Complete User Journey

To tie every chapter together, let's now walk through a complete journey the way a real library member would experience it, explaining at each step exactly what's happening behind the scenes.

16. A member opens the site for the first time → View: `catalog_views.home` → sees a welcome page with general statistics (without being logged in).
17. The member clicks "Explore" in the navbar → View: `book_list_html` → sees every available book, and can search by title or author.
18. The member tries to borrow a book → Django automatically redirects them to the login page (`@login_required`), since they aren't signed in yet.
19. The member doesn't have an account, so they click "Sign Up" → View: `signup_view` → fills in the `UserCreationForm` → if the data is valid, an account is created in the `auth_user` table, and they're redirected to the login page.
20. The member logs in → `CustomLoginView` → Django creates a new session and redirects them to the catalog page (`LOGIN_REDIRECT_URL`).
21. The member clicks "Borrow" on an available book → View: `borrow_book` → checks there's no existing loan for the same book, checks copies are available, creates a new `Loan`, decrements `available_copies`, and redirects to the "My Books" page.
22. The member sees their book on the `my_loans` page → View: `my_loans` → queries every one of their `Loans` with status `BORROWED`.
23. Once they've finished the book, they click "Return" → View: `return_book` → updates the status to `RETURNED`, records `return_date`, and increments `available_copies` again.
24. At any point, the library manager (Admin) can open `/admin/` to see every book and loan, add new books, or use the "Add 1 copy" action when a new shipment of books arrives.

Note:

Notice how `available_copies` is the "backbone" connecting three different operations: borrowing (decrements it), returning (increments it), and admin actions (increment or decrement it manually). Any future change to inventory logic needs to account for all three places at once, so the numbers stay accurate.

Chapter 20: Notes and Suggested Improvements

The project we've walked through works and covers all the core functionality, but like any learning project, there are a number of things that could be improved on the road to something closer to production. The list below is organized as suggested exercises for the reader:

1. Security and settings

- Move SECRET_KEY to an environment variable using a library such as python-decouple or django-environ, instead of hard-coding it in the source.
- Make DEBUG driven by an environment variable, and default it to False everywhere except local development.
- Set ALLOWED_HOSTS to the actual domain names instead of leaving it empty.

2. Cleaning up the settings

- LOGIN_REDIRECT_URL and LOGOUT_REDIRECT_URL are duplicated in settings.py — consolidate them into a single place near the end of the file.
- Add a proper requirements.txt with pip freeze > requirements.txt so anyone can recreate the same environment easily.

3. Models

- The Book model has two image fields (cover and cover_image) uploading to two different folders (covers/ and book_covers/) — merging them into one field simplifies the code and removes ambiguity, especially since home.html relies on cover_image while the rest of the pages rely on cover.
- If you want a configurable loan period instead of the hard-coded 14-day value inside loans/models.py, you could move it into a setting in settings.py.

4. Expanding the API

- Add an API endpoint for returning books (api/return/<loan_id>/), following the same pattern as BorrowBookAPIView, so the API is complete and any external client can perform every operation without falling back to the HTML pages.
- Actually enable drf_spectacular's documentation by registering these routes in the main urls.py:

```
from drf_spectacular.views import (
    SpectacularAPIView, SpectacularSwaggerView
)

urlpatterns += [
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema'),
```

```
name='swagger-ui'),  
]
```

- Add a `REST_FRAMEWORK` setting in `settings.py` specifying `DEFAULT_SCHEMA_CLASS` as `drf_spectacular.openapi.AutoSchema` so `drf_spectacular` works fully.

5. Tests

- The `tests.py` files across all three apps (`accounts`, `catalog`, `loans`) are still empty. A great first step: write a test confirming that borrowing an available book decrements `available_copies` by one, and a test confirming that borrowing an unavailable book raises a `ValidationError`.

6. User experience (UX)

- `accounts/login.html` doesn't extend `base.html` (no `{% extends %}`), unlike every other template — unifying it would let it automatically inherit the same design and navbar.
- `static/project.js` contains signup form validation code that isn't actually used in the current `signup.html`, since that page relies directly on the Django Form instead. Review the file and remove any unused code to avoid confusion later.

7. Ideas for expanding the project further

- A loan renewal system, so a member doesn't have to return and re-borrow the same book.
- A simple fines system for books overdue past the 14-day window.
- A ratings and reviews page for each book.
- An API permissions system distinguishing a regular member from a librarian.
- Pagination on the book list API once the number of books grows large.

Conclusion

If you've made it here and worked through every step, you now have a complete understanding of a real Django project — from the very first project settings, all the way to the smallest detail of the API. More importantly, you've seen how small decisions in the code (like the `clean()` method on a model, or using `get_object_or_404`) protect the project from a whole class of bugs and vulnerabilities, without needing extra, complicated code.

The single most important piece of advice: don't just read. Close the book, and try building your own version following the same steps, without looking back at the original code unless you get stuck. Then try applying the improvements listed in the previous chapter, one at a time, and each time ask yourself: "How will this affect the rest of the system?" Questions like that are exactly what separates a programmer who writes code that works from one who builds systems.

Note:

If you have questions while working through this, these resources will help a lot: the official Django documentation (docs.djangoproject.com), the Django REST Framework documentation (www.django-rest-framework.org), and the error message Django itself prints in the terminal — it usually tells you exactly where the problem is.

Appendix: the most commonly used commands

Command	Purpose
<code>django-admin startproject <name> .</code>	Create a new Django project
<code>python manage.py startapp <name></code>	Create a new app inside the project
<code>python manage.py makemigrations</code>	Generate migration files based on model changes
<code>python manage.py migrate</code>	Actually apply migration files to the database
<code>python manage.py createsuperuser</code>	Create an admin account for accessing the admin panel
<code>python manage.py runserver</code>	Start the local development server
<code>python manage.py shell</code>	Open an interactive shell with the full Django environment loaded
<code>pip freeze > requirements.txt</code>	Generate a file listing every installed package and its version